

1. * Wrong sign flag causes incorrect positive/negative result interpretation
* Wrong zero flag may falsely indicate zero output
* proper flag updating and signed arithmetic checking ensure correct subtraction results
-

2. * Memory-based operands requires extra access cycles, causing slower execution and update delays.
* Using register operands improves speed and maintains accurate arithmetic calculation
-

3. * Repeated NOT operations continuously flip bits, causing loss of the original signal state
* Without reference values, the controller may misinterpret signals and make wrong decisions.
* store the original signal or use a status/reference register to maintain correct control logic.
-

4. * Incorrect AND masking can remove valid signal bits along with noise, causing data loss and wrong output.
* choosing the correct mask value preserves required bits while filtering unwanted noise accurately.
-

5. *Ignoring carry and overflow causes cumulative arithmetic errors.
*Incorrect values propagate through continuous operation.
*Regular flag checking and error validation ensures accurate calculations.

6. *Incorrect handling of arithmetic instructions and flags causes overflow and wrong results
*Large or memory based operands reduce execution accuracy and increase errors.

7. *XOR operation is reversible; so applying the same key restores the original data
*Using different keys during encoding and decoding produces incorrect outputs.
*Store and apply a consistent key for both operation to ensure accurate data transformation.

8. *Improper OR operation can activate outputs unintentionally, when any overlapping input becomes HIGH
*Correct logical conditions and proper input validation prevent unwanted control outputs.

9. * Increment beyond limit causes data overwrite
* Wrong decrement may skip or lose data
* Improper flags create overflow/underflow errors
* Boundary checking prevents invalid access
* Circular buffering and correct flags ensure reliable buffer management.

10. * Improper data dependency handling causes incorrect data usage between operations.
* Wrong instruction order produces incorrect outputs and unstable system behavior.
* Uncontrolled execution reduces processing efficiency and increases delays.
* Proper sequencing and synchronization ensure correct data flow between steps.
* Using dependency checking and controlled instruction execution improves reliability and efficiency.

11. Same as 9th question

12. * Improper interaction between arithmetic and logic operations causes incorrect outputs.
* Invalid intermediate values reduce processing accuracy.
* Inefficient instruction execution increases processing time.
* Validation of intermediate results ensures correct data flow.
* Proper instruction sequencing and optimized logic operations improve accuracy and efficiency.

13. * Incorrect rotate operations cause wrong bit positioning in registers.
- * Improper rotate direction produces inconsistent device response
 - * Wrong carry flag handling leads to unpredictable outputs.
 - * Validation of intermediate register values ensures correct bit manipulation.
 - * Controlled rotate operations with proper carry management improves accurate and reliable device.
-

14. * Inefficient instruction cycles increase execution time and latency.
- * Frequent memory access reduces processing speed and resource utilization.
 - * Instruction dependencies create delays in execution flow.
 - * Effective register usage and optimized instruction sequencing improve efficiency.
 - * Reducing memory access and applying parallel / pipelined execution enhances throughput and accuracy.
-

15. * Confusing rotate and shift operations cause incorrect bit movement.
- * Repeated wrong rotations lead to data corruption during transmission.
 - * Improper carry flag handling creates unpredictable outputs.
 - * Correct distinction between shifts and rotate operations ensures proper bit positioning.
 - * Tracking carry flag changes and validating rotated data improves transmission accuracy and reliability.

16. * Poor instruction scheduling increases latency and reduces responsiveness.
* Inefficient workload distribution create execution bottleneck.
* Unbalanced resource usage lowers processing efficiency.
* proper instruction ~~the~~ scheduling and parallel execution improve performance.
* Load balancing and optimized resource utilization enhance efficiency and scalability.
-

17. * Wrong address resolution fetches incorrect data
* CPU-memory timing mismatch cause invalid operations.
* proper address checking and synchronized execution prevent data corruption.
-

18. * Direct addressing gives fast execution with low delay.
* Indirect ~~addressing~~ addressing supports flexible and scalable memory access.
-

19. * Arithmetic operation update status flags such as zero, carry and sign flags.
* Program decisions use these flags to control branching and execution flow.
* Accurate flag updates ensure correct and reliable program behaviours.
-

20. * Incorrect updating of status flags after arithmetic operations causes wrong conditional execution.

* Faulty ALU operations or improper status register handling may produce incorrect flag values.

21. * The processor reads the address field from the instruction.
* That address is used to access memory and obtain the effective address.
* The effective address is then used to fetch the actual operand for execution.

22. * Opcode defines the operation and the address field locates the operand in memory.
* CPU components execute the instruction through fetch, decode and execute stages.

23. * Use essential flags like zero, carry, sign and overflow for accurate decision-making.
* Avoid unnecessary flags to reduce hardware complexity.
* Choose flags based on system requirements.

24. * More flags improves condition monitoring and decision accuracy.
* Excessive flags increases hardware complexity, reducing performance and scalability.

25. * Register instructions operates directly on CPU register without memory access.
- * They reduce execution time and improves processing speed.
 - * Fewer memory accesses decrease latency and increase efficiency
 - * Arithmetic and logic operations are performed quickly using accumulators and general-purpose register.
 - * These instructions improves overall CPU performance and efficiency improve.

26. * Direct addressing provides faster execution with low overhead.
- * Direct addressing supports only limited memory range.
 - * Indirect addressing allows flexible access to large and complex data structure.
 - * Indirect addressing requires extra memory access, increasing execution time.
 - * Choosing the addressing mode depends on balancing speed, flexibility and scalability.

27.

Instruction Fetch



Instruction Decode



Register selection



ALU operation



Store Result
and
Update Flags

28. * Extended register operations perform's fast arithmetic and logic operation inside CPU register.

* Indirect addressing accesses dynamic data efficiently

* Register stores temporary data and addresses for faster execution

* Indirect addressing improves memory flexibility.

* Combining both improves system speed, flexibility and efficient data handling

29. * Jump and call instructions provides flexible program control and modular design.
- * Excessive jumps reduce program readability and increase complexity.
 - * Frequent call instructions introduce execution overhead and memory usage.
 - * Proper control flow improves maintainability and debugging.
 - * Balanced use of jump and call instructions ensure efficient and organised execution.
-

30. * Stack organization supports function calls, recursion and interrupt handling efficiently.
- * It provides structured and organized program execution flow.
 - * Excessive stack usage increases memory consumption.
 - * Deep recursion may cause stack overflow and reduce reliability.
 - * Proper stack management improves performance, scalability and efficient memory usage.
-

31. *The CALL instruction stores the return address in the stack.

*The stack pointer is updated after saving the address.

*Then program control moves to the subroutine

*The RETURN instruction retrieves the saved address from the stack

*The processor restores the address and resumes normal program execution.

32. *PUSH operation stores data into the stack.

*During PUSH, the stack pointer moves to the next stack location.

*POP operation retrieves data from the stack.

*During POP, the stack pointer returns to the previous location.

*Proper stack pointer movement ensures correct data storage and execution flow.
